API gateway

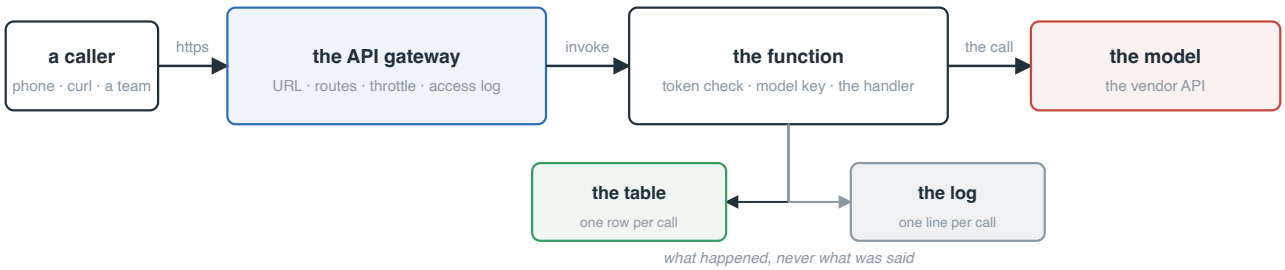
A one-page primer: a public address for a function, and what sits on either side of it

> Written for the *publish the API* lab. The platform in the course is AWS; the mechanism is the same on every cloud. Names are October 2026 and will change.

What it is

An **API gateway** is the thing that faces the internet so that your function does not have to. It owns a public address, decides which route a request is for, applies whatever rules you set at the edge (a throttle, an authorizer, a size limit), hands the request to the function, and returns the function's answer. It keeps an access log. It holds no code of yours and no secret of yours. In the lab:

the address	`https://<id>.execute-api.us-east-1.amazonaws.com`, public, no signature required
the routes	one, `$default`: every path and method goes to the function, which routes by path itself
the integration	Lambda proxy, payload version 2: the request arrives as one JSON event, the same shape a function URL sends
the throttle	a rate and a burst on the stage; above them the gateway answers `429` and the function never runs
the access log	one line per request at the edge, separate from the function's own log



One request. The gateway owns the address; the function owns the secret; the table and the log own the record.

What sits on either side

- **Before the gateway:** anyone. A phone on mobile data, a classmate's laptop, a script. The gateway is the first thing in the course that says no to a request, and it says it before anything of yours runs.
- **Behind the gateway:** the function, with the model key in its environment and the token check in its code. The gateway can be given an authorizer of its own, and in a larger system it is; in the lab the check is in the function so that you can read it.
- **Beside the function:** the table and the log. Every call writes one row to the table, what happened and never what was said, and one line to the function's log. The gateway's access log says that a request arrived and what status left; the function's log says what it did in between.

What the model changes

Nothing about the gateway is specific to a model. What changes is what sits behind it: a call that takes a second, costs money per request, and holds a key that pays for every call made with it. So the throttle at the edge is the first rate limiter in the course, protecting the one thing behind it that costs; the token exists so that a public address does not mean a public model key; and the row the function writes is the inference table from the lectures, the record every later measurement reads.

What it is not

It is not a load balancer, though the two are often drawn in the same box: the gateway routes by path and applies rules, and the function platform does the scaling. It is not where the model key goes. And it is not the record of what the system did: that is the table and the log, which the gateway never sees.

As of October 2026

AWS has two kinds: the **HTTP API**, which the lab uses, cheap and simple, with the `$default` route and the throttle on the stage; and the older **REST API**, with more features (request validation, caching, usage plans with API keys) and more steps to set up. A function URL is a gateway with one route and no rules; the Learner Lab refuses anonymous ones, which is why the lab uses the gateway. Google Cloud's API Gateway and Azure's API Management are the same component.